

WeMips: Online MIPS Emulator Guide

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

What is WeMips?

WeMips is an online MIPS emulator that runs directly in your web browser. It provides a convenient way to write, execute, and debug MIPS assembly programs without installing any software. The emulator runs your programs step-by-step and can be interrupted by errors, input requests, or code completion.

Access WeMips Online:

Visit the WeMips emulator at:

<https://wemips.ralfgerlich.biz/>

1 Features

1.1 Constant Expressions

Wherever an immediate value is required, you can use expressions with operands such as addition, subtraction, etc., as long as the result is a constant value.

For example, it is possible to use:

```
addi $t0, $zero, (15+3*32)
```

Operators are evaluated according to their precedence. Addition and subtraction have lower precedence than multiplication, and the former are thus evaluated after the latter. Operands are listed in decreasing order of precedence (operators further down in the list are evaluated after operators further up):

- Unary Plus (“+”), Unary Minus (“-”), Bitwise NOT (“~”), Logical NOT (“!”)
- Multiplication (“*”), Division (“/”), Remainder (“%”)
- Addition (“+”), Subtraction (“-”)
- Shift Left (“<<”), Arithmetic Shift Right (“>>”), Logical Shift Right (“>>>”)
- Less Than (“<”), Less Than or Equal (“<=”), Greater Than or Equal (“>=”), Greater Than (“>”)
- Equals (“==”), Not Equals (“!=”)
- Bitwise AND (“&”)

- Bitwise XOR (“^”)
- Bitwise OR (“|”)
- Logical AND (“&&”)
- Logical OR (“||”)
- Ternary Conditional Operator (“cond?trueValue:falseValue”)

1.2 Extracting 16-bit Values

It is also possible to extract the lower and higher 16 bits of constants, respectively, by using the `lo16` and `hi16` functions:

```
lui    $t0, hi16(123456)
addiu  $t0, $t0, lo16(123456)
```

1.3 Numerical Constants

Numerical constants can be provided in decimal, hexadecimal, octal, and binary:

- **Hexadecimal numbers** are specified with a `0x` or `0X` prefix, e.g., `0xabcd`. Case does not matter, i.e., `0xABcd` is the same as `0xabCD`.
- **Octal numbers** are prefixed by a `0`, e.g., `0775`.
- **Binary numbers** are specified with a `0b` or `0B` prefix, e.g., `0b101001010`.
- **Decimal numbers** are specified without any prefix. That also means that they may *not* start with a zero (`0`), otherwise they will be interpreted as octal (which will lead to errors if using non-octal digits).

2 Register View

WeMips provides a register view where you can:

- Toggle which registers are currently visible (S, T, A, or V registers)
- View a register’s current value
- Modify register values by clicking on them

Registers are initially populated with junk data until an instruction modifies them. A register is composed of 32 bits and can therefore hold 2^{32} different values.

3 Available MIPS Instructions

WeMips supports a comprehensive set of MIPS instructions. Instructions shown in **bold** are hardware instructions; all others are pseudo-instructions.

Note: The floating point unit (FPU) is not implemented!

Category	Instructions
Arithmetic (signed)	ADD, ADDI, SUB, MULT, DIV
Arithmetic (unsigned)	ADDU, ADDIU, SUBU, MULTU, DIVU
Comparison Instructions	SLT, SLTI, SLTU, SLTIU
Logical Operations	AND, ANDI, OR, ORI, XOR, XORI, NOR
Shift Operations	SLL, SLLV, SRL, SRLV, SRA, SRAV
Jump Instructions	B, J, JAL, JR, JALR
Branch Instructions	BEQ, BNE, BGT, BGE, BLT, BLE
Branch (unsigned)	BGTU, BGEU, BLTU, BLEU
Branch (zero comp.)	BEQZ, BNEZ, BGTZ, BGEZ, BLTZ, BLEZ
Branch and Link	BGEZAL, BLTZAL
Register Transfer	MFHI, MFLO, MTHI, MTLO
Load Instructions	LB, LBU, LH, LHW, LW, LWL, LWR, LUI
Store Instructions	SB, SH, SW, SWL, SWR

Table 1: Available MIPS Instructions in WeMips

4 Available MIPS Syscalls

The syscalls supported are similar to those used by the MARS emulator. Common syscalls include:

Service	Code (\$v0)	Arguments
Print Integer	1	\$a0 = integer to print
Print Float	2	\$f12 = float to print
Print Double	3	\$f12 = double to print
Print String	4	\$a0 = address of null-terminated string
Read Integer	5	Returns integer in \$v0
Read Float	6	Returns float in \$f0
Read Double	7	Returns double in \$f0
Read String	8	\$a0 = buffer address, \$a1 = max length
Allocate Heap Memory	9	\$a0 = bytes to allocate, returns address in \$v0
Exit	10	Terminates program
Print Character	11	\$a0 = character to print
Read Character	12	Returns character in \$v0

Table 2: Common WeMips System Calls

5 Example Program

Here's a simple example to demonstrate WeMips usage:

```
1  # Simple addition and output program
2  .text
3
4  main:
5      addi $t0, $zero, 10
6      addi $t1, $zero, 20
7      add  $t2, $t0, $t1
8      addi $v0, $zero, 1
9      add  $a0, $t2, $zero
10     syscall
```

Note: WeMips runs in your browser and provides immediate feedback. You can step through instructions, view register changes, and debug your code interactively.